A Stochastic Dynamic Programming Approach for Request Acceptance and Unsplittable Scheduling Decisions under Uncertainty

Marvin Caspar(✉)[0000−0003−1025−9772], Konstantin Kloster[0000−0002−2538−9900], and Oliver Wendt[0000−0002−7102−3141]

Business Information Systems & Operations Research (BISOR),
RPTU Kaiserslautern-Landau, Postfach 3049,
Erwin-Schrödinger-Str., 67653 Kaiserslautern, Germany
{marvin.caspar,konstantin.kloster,wendt}@wiwi.uni-kl.de
https://wiwi.rptu.de/fgs/bisor

Abstract. The trade-off between gaining revenue through accepting a random incoming request and its scheduling against a higher potential revenue in the future needs sophisticated deliberations and decision-making. To gain deeper insights and to examine such decisions under uncertainty, request acceptance and scheduling should be made simultaneously. This paper introduces the *Dynamic Request Acceptance and Unsplittable Scheduling Problem* (DRAUSP), where we consider a single resource with a given capacity across multiple time slots. In each decision period of a finite time horizon, a single request arrives, and the objective is to either accept or reject the request and, if accepted, simultaneously schedule it onto available time slots without exceeding the capacity to maximize revenue. We formulate the DRAUSP as a *Markov Decision Process* and present a *Stochastic Dynamic Programming* algorithm to solve it optimally. In addition, we introduce a *Dimension Compression* algorithm to compute upper and lower bounds. In our experiments, we demonstrate the effectiveness of these bounds and also show that simple heuristics, such as a *First Come First Serve* heuristic, are not sufficient to cope with the complexity of the DRAUSP. Therefore, more sophisticated heuristics become necessary, since well-planned acceptance and scheduling decisions are essential for revenue maximization.

Keywords: Dynamic Optimization · Request Acceptance and Scheduling · Revenue Maximization · Stochastic Dynamic Programming.

1 Introduction

A central task of revenue management is to manage the trade-off between receiving immediate revenue for selling a fixed and perishable capacity or product against a higher potential revenue in the future [21]. However, determining revenue-maximizing policies is particularly challenging in complex and competitive environments, where disruptions, unforeseen events, and other uncertainties

make predicting the required capacity difficult. Airlines and hotels, for example, often face sudden diverse incoming requests, forcing them to make decisions about accepting or rejecting such requests on short notice to maximize their revenue (see [8]). In addition, such decisions may also affect subsequent scheduling tasks. Scheduling problems arise when scarce resources must be allocated to a set of tasks while optimizing some objective, and are widespread across industries [13].

In this paper, we study the *Dynamic Request Acceptance and Unsplittable Scheduling Problem* (DRAUSP), where the task is to schedule accepted requests onto the limited capacity of a single resource across different time slots. For this problem, we distinguish between an upfront decision horizon, during which $T_d \in \mathbb{N}_{>0}$ requests arrive successively, and a subsequent service period, where the accepted requests get fulfilled. The service period is divided into $K \in \mathbb{N}_{>0}$ discrete time slots, each with a capacity $C_k \in \mathbb{N}_{>0}$, where $k = 1, ..., K$. Each request is defined by a tuple $[r, q]$, where r represents a positive revenue yielded upon acceptance, and $q = (q_1, ..., q_{len(q)}) \in \{0, 1, ..., C_k\}^{len(q)}$ is a discrete, non-negative vector with length $len(q) \in \{1, 2, ..., K\}$ representing the demanded capacity per time slot. Upon acceptance, requests must be scheduled onto the different time slots such that all accepted requests can be fulfilled non-preemptively during the service period without exceeding the capacity C_k of any time slot k. Depending on the available capacity and the length of a request $len(q)$, there are up to $K - len(q) + 1$ possibilities to schedule the request. At each of the T_d periods of the decision horizon, a single request is independent and identically drawn from a stationary distribution over a finite set of possible requests N. The decision to accept or decline the current request, along with its scheduling if accepted, must be made immediately, i.e., before the next request arrives. The objective of the DRAUSP is to maximize the total revenue over the decision horizon.

Figure 1 shows an example with $K = 6$ time slots and a capacity of $C_k = 5$ for these time slots $k = 1, ..., K$. In the current time step of the decision horizon, a request with a certain revenue r and a demand vector $q = (1, 2, 2, 1)$ arrives, and we have to decide whether to accept this request and, if so, on which time slots to schedule it. Since several requests have already been accepted in this example we cannot schedule this request to time slots $[1; 4]$ due to insufficient capacity, while possible time slots are $[2; 5]$ and $[3; 6]$. In this example, we decide to accept the request and schedule it into time slots $[3; 6]$. However, the request arriving in the next time step of the decision horizon with demand vector $q = (2, 1, 1)$ cannot be accepted, as it would exceed the capacity.

The DRAUSP has practical applications including job acceptance and scheduling decisions on high-performance computing clusters, where jobs might be non-preemptive and may have different demand profiles over time. However, defining and describing the problem alone is not sufficient for practical use; we also need to develop solution approaches. Consequently, we present a *Stochastic Dynamic Programming* (SDP) algorithm to solve the DRAUSP optimally. In addition, we introduce a *Dimension Compression* algorithm to compute lower and upper bounds on the optimal solution.

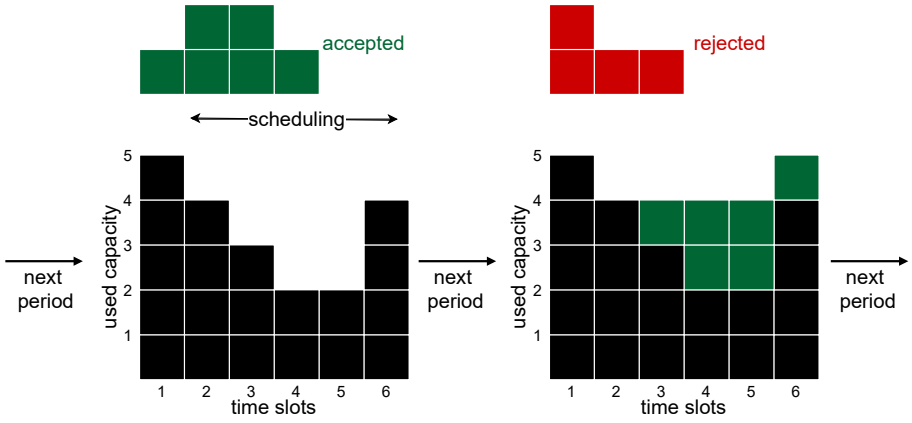

Fig. 1. An illustration of the DRAUSP with $K = 6$ time slots and an initial capacity of $C_k = 5$ for these time slots $k = 1, ..., K$. The first request with $q = (1, 2, 2, 1)$ gets accepted and scheduled onto time slots $[3; 6]$ (time slots $[2; 5]$ would also be possible, while $[1; 4]$ is not possible). The following request in the next decision period with $q = (2, 1, 1)$ cannot be accepted due to insufficient capacity.

The remainder of this paper is structured as follows. Section 2 interlinks the DRAUSP with the relevant literature and distinguishes it from similar problems. In Section 3, we formalize the problem as a *Markov Decision Process* (MDP). Afterward, we present an SDP algorithm for solving the DRAUSP optimally and demonstrate how to obtain lower and upper bounds using a DC algorithm in Section 4. Section 5 provides detailed results of our computational experiments, which we conducted to analyze the tightness of the DC bounds and to compare our SDP with a *First Come First Serve* (FCFS) heuristic. Finally, we conclude this work and discuss future research implications in Section 6.

2 Related Literature

Request acceptance decisions under uncertainty have already been addressed in the revenue management literature, particularly in the context of *Airline Revenue Management* (ARM). Numerous papers, such as [6, 18, 3], consider request acceptance decisions within ARM, incorporating elements like different fare classes, random cancellations, and overbooking. These studies utilize *Reinforcement Learning* (RL) to obtain approximate solutions. Similar problems can also be found in *Cargo Revenue Management*. Seo et al. [17], for example, investigate request acceptance decisions of ad-hoc and contractual containers in a sea cargo revenue management environment, using SDP and RL to solve the problem.

Schwind and Wendt [16] study the less constrained problem of maximizing revenue by accepting or rejecting stochastic requests that arrive dynamically

during an upfront decision horizon, subject to the capacity constraints of one or more resources. To solve this problem, they introduced an RL algorithm and benchmarked its performance against optimal solutions obtained by SDP. The latter problem is closely related to the *Dynamic and Stochastic Knapsack Problem with Deadlines* [12] that belongs to the class of *Dynamic and Stochastic Knapsack Problems* (DSKP) (see [9] and references therein), where a capacitated knapsack is given, and items arrive sequentially as value-size pairs. Each item must either be irrevocably accepted or rejected to maximize the total value of items without exceeding the knapsack's capacity. Yang et al. [24] consider a multidimensional knapsack with the objective of maximizing the total value while respecting the capacities of the different dimensions, and introduce two online algorithms using reservation functions to solve this problem.

While all of the papers mentioned so far study request acceptance decisions, they do not incorporate scheduling considerations. In contrast, the class of *Online Scheduling Problems* (OSP) addresses scheduling decisions without the prior step of accepting or rejecting requests (see [14]). The *Stochastic Resource Constrained Project Scheduling Problem* (SRCPSP) belongs to this OSP class, where a set of jobs with random processing times, precedence constraints, and different resource consumption requirements is scheduled across multiple resources, minimizing a project cost function or the project makespan. Stork [20] employs different scheduling policies and various branch-and-bound algorithms to solve the SRCPSP.

The *Unsplittable Flow Problem* (UFP) is a unifying framework for a variety of scheduling and load-balancing problems such as the *Interval Scheduling Problem* [10], *Storage Allocation Problem* [11] and the *Geometric Strip Packing Problem* [5], that is closely related to the DRAUSP. In the UFP with bag constraints [4], there is a resource with a capacity varying over time and a set of jobs. Each job is specified by a certain demand that is constant over time, a profit, and a set of job instances. The job instances define different possible time intervals for each job, and the objective of the UFP with bag constraints is to select at most one job instance for each job such that the profit is maximized. In contrast to the offline case, literature covering online algorithms for UFP scheduling, to the best of our knowledge, is limited to special variants (see, e.g., [7]), and even for those, competitive analysis does not yield promising bounds.

A research stream that deals with request acceptance and scheduling decisions under uncertainty in the context of manufacturing is the *Stochastic Order Acceptance and Scheduling Problem* (SOASP) [19]. In the SOASP, decisions about which orders to accept for processing and how to schedule them on machines must be made jointly and in real time. Usually, in such SOASPs, accepted orders must be scheduled according to their processing times and due dates and are not subject to capacity restrictions as in the DRAUSP. Xu et al. [23] present a SOASP with sequence-dependent setup times and stochastic orders with fixed revenues for certain order types. The authors formulate the problem as an MDP and develop an order acceptance rule based on opportunity costs, which is com-

pared to a *First Come First Accept* heuristic and the optimal solutions of the corresponding deterministic problems with full information.

Finally, there are several related streams of research in the class of *Online Allocation Problems* (OAP), where real-time decisions about the allocation of accepted requests subject to resource constraints must be made (see [1] for related OAP literature) and therefore our DRAUSP can also be included in this problem class. Balseiro et al. [1] studied a generic OAP with resource constraints where resource consumption can be allocated to resources as required, which is not a fact for the DRAUSP. They present a numerical study for an online linear program with stochastic inputs where action vectors control the allocation of different resources. The authors propose a dual mirror descent algorithm to solve the OAP and compare the results with the hindsight optimum.

The DRAUSP with request acceptance and scheduling decisions under uncertainty is closely related to different problem categories such as ARM problems, DSKPs, OSPs, UFPs, SOASPs, and OAPs. However, to the best of our knowledge, no literature treats such request acceptance decisions of incoming requests with stochastic demand and revenue over a finite time horizon and simultaneous unsplittable scheduling on a resource over different time slots to maximize the aggregate revenue. Thus, we address this research gap with the DRAUSP and provide an MDP formulation in the following section.

3 Markov Decision Process

Recall that in the DRAUSP, $T_d \in \mathbb{N}_{>0}$ requests, independently and identically drawn from a stationary distribution over a finite set of possible requests N, arrive sequentially over a finite time horizon. Each request $[r, q]$ must either be accepted or rejected and, if accepted, non-preemptively scheduled onto $K \in \mathbb{N}_{>0}$ different time slots with capacities $C_k \in \mathbb{N}_{>0}$, where $k = 1, ..., K$. In this section, we introduce an MDP model for the DRAUSP. To this end, we define $T = \{0, ..., T_d\}$ as the time horizon for the MDP. Besides that, an MDP consists of a set of states, a set of actions, a reward function, and the state transitions [15]. We define these elements in the following paragraphs.

States: The state space $S = \{0, 1, ..., C_1\} \times ... \times \{0, 1, ..., C_K\}$ consists of all possible combinations of remaining capacities. We use $s_t \in S$ to represent the state in period t, where $t \in \{T_d, ..., 0\}$ is the down-counting index of periods indicating the number of remaining requests. Furthermore, we define the initial state $s_{T_d} = (C_1, ..., C_K)$, where all time slots have their initial capacity C_k, and the terminal state s_0, where no more requests arrive. Finally, we use $cap \in S$ to refer to the currently available capacity, i.e., $s_t = cap$.

Actions: Within each decision period $t \in T \backslash \{0\}$, a random request $[r, q] \in N$ arrives in the current state s_t and determines the actual action space A_s. This set A_s contains actions represented by tuples $a = [a_r, a_q]$ consisting of a revenue $a_r \in \{0, r\}$, which is obtained when the action is chosen, and a vector $a_q \in Q_a$ of length K, which specifies the required demand for all K time slots. The set $Q_a = \{\vec{0}, (q_1, ..., q_{len(q)}, 0, ..., 0), (0, q_1, ..., q_{len(q)}, 0, ..., 0), ..., (0, ..., 0, q_1, ..., q_{len(q)})\}$ with

$|Q_a| = K - len(q) + 2$ results from scheduling the demand vector q onto all possible time slots. Q_a also includes $\vec{0}$ to represent the reject action $a = [0, \vec{0}]$, which yields no revenue and demands no capacity.

Transitions: Between two states s_t and s_{t-1} with $t \in T \setminus \{0\}$ a transition occurs, which depends on the selected action $a = [a_r, a_q] \in A_s$. However, we only consider actions that do not exceed the capacity, i.e., actions for which $cap - a_q \geq 0$ holds since we do not allow overbooking. For all valid actions, including the reject action $[0, \vec{0}]$, the state transition principle for the DRAUSP becomes $s_{t-1} = cap - a_q$. Reaching the terminal state s_0 signifies the end of the decision process, as no further requests arrive and no more transitions occur.

Rewards: Transitioning to the next state by choosing an action $a = [a_r, a_q]$ yields the reward a_r in state s_t, where $t \in T \setminus \{0\}$. The reward is thus defined by the reward function $R(s_t, a) = a_r$.

Policy: A policy $\pi(s_t) : S \longrightarrow A_s$ is a function indicating which action to take in each state. In the DRAUSP, the objective is to find the optimal policy, i.e., the policy giving us the optimal action in each state to maximize the cumulative reward over the time horizon T. Thus, the objective function of the DRAUSP becomes:

$$\max_{\pi(s_t)} \mathbb{E} \left\{ \sum_{t \in T \setminus \{0\}} R(s_t, \pi(s_t)) \right\} \tag{1}$$

Value function: The value function $V_t(cap)$ can be used to calculate the expected objective function value when starting in state $cap \in S$ at time step $t \in T$ and choosing the best action until the terminal state s_0 is reached at $t = 0$. Given that $Pr([r, q])$ represents the probability that the request $[r, q] \in N$ arrives in period t, the calculation of $V_t(cap)$ can be expressed as:

$$V_t(cap) = \begin{cases} -\infty & \forall cap \notin (\mathbb{R}_0^+)^K, t \in T \\ 0 & \forall cap \in (\mathbb{R}_0^+)^K,\ t = 0 \\ \sum\limits_{[r,q] \in N} Pr([r, q]) \max\limits_{[a_r, a_q] \in A_s} \{a_r + V_{t-1}(cap - a_q)\} & \text{else} \end{cases} \tag{2}$$

4 Solution Methods for the DRAUSP

4.1 Stochastic Dynamic Programming

One method to solve the value function of the DRAUSP presented in Section 3 is SDP. In this section, we present a forward recursion SDP algorithm, which has the advantage that we only need to compute the values of states that are actually needed, i.e., we do not compute the values for the complete state space.

The function `recursive_sdp` in Algorithm 2 takes the number of decision periods T_d, the vector of initial capacities cap, and the set of requests N as input. For each decision period t, where $t = 1, ..., T_d$ represents the number of remaining requests, we compute the optimal expected value $V_t^\star$ for the initial capacity cap

Algorithm 1: Value computation (for Algorithm 2)

```
1  Function get_value(t, cap, N, Ω):
2      if t = 0 then
3          return 0
4      end
5      if Ω contains [t, cap] then
6          return Ω[t, cap]
7      end
8      value ← 0;
9      foreach [r, q] ∈ N do
10         best_action_value ← −∞ ;
11         A_s = get_action_space(r, q, cap)
12         foreach [a_r, a_q] ∈ A_s do
13             new_state ← cap − a_q;
14             if new_state is valid then
15                 action_value ← a_r + get_value(t − 1, new_state, N, Ω)
16             end
17             else
18                 action_value ← −∞
19             end
20             if action_value > best_action_value then
21                 best_action_value ← action_value
22             end
23         end
24         value ← value + best_action_value;
25     end
26     Ω[t, cap] ← value/|N|
27     return value/|N|
```

when there are t incoming requests remaining by calling the function `get_value`. Throughout the computation, we utilize a hashmap Ω to memoize computed values. Finally, the function returns $V^{\star}_{T_d}$, which is the optimal expected value obtainable when starting with the initial capacity *cap* and facing T_d requests.

Within the `get_value` function in Algorithm 1, we select the best action for each request in the current state by recursively calling the `get_value` function and update the expected value according to equation (2), assuming a discrete uniform distribution, where each request is equally likely to arrive in a period.

When using a thread-safe hashmap, this algorithm can be accelerated by parallelizing the for-each loop over the requests (see line 9 of Algorithm 1) within the `get_value` function. However, this loop should only be parallelized when the `get_value` is called from the `recursive_sdp` function, i.e., not when the function `get_value` is called recursively.

Algorithm 2: Forward recursion stochastic dynamic programming for the DRAUSP

```
1 Function recursive_sdp(T_d, cap, N):
2 |   Ω ← ∅ ;                                          // init hashmap
3 |   for t ← 1 to T_d do
4 |   |   V_t^⋆ ← get_value (t, cap, N, Ω) ;          // see Algorithm 1
5 |   end
6 |   return V_{T_d}^⋆
```

4.2 Dimension Compression Algorithm

SDP algorithms, like the one presented in Section 3.1, can quickly become computationally intractable due to the curse of dimensionality [2]. To overcome this problem, heuristics are frequently used to find good, though not necessarily optimal, solutions. In this section, we present a *Dimension Compression* (DC) algorithm to compute lower and upper bounds for the DRAUSP (see Algorithm 3), which may be used to evaluate the quality of heuristics if the SDP cannot be solved anymore. The main idea of the DC algorithm is to reduce the number of time slots K and the lengths of the demand vectors q of all requests $[r, q] \in N$. To be more precise, we halve the number of time slots K and the lengths of the demand vectors. The reduced problem can then be solved using the `recursive_sdp` function.

We start by checking if K is an even number. If not, we take the last element of the capacity vector *cap* and append it to *cap*. We also increase K by one. Afterward, we create two vectors $\overline{cap}$ and $\underline{cap}$ of length K/2, which will represent our new capacity vectors to compute the upper and the lower bound, respectively. To compute $\overline{cap}$, we look at all non-overlapping pairs of consecutive elements in *cap* and take the maximum value of each pair multiplied by two. Similarly, we take the minimum value of each non-overlapping pair of consecutive elements in *cap* and add it to $\underline{cap}$.

In the next part of the algorithm (see lines 13 - 23 of Algorithm 3), we compute the new demand vectors. We iterate through all requests $[r, q]$ of the original request set N. If the length of the current demand vector q is not even, we will append 0 to q. Then, we create the vectors $\overline{q}$ and $\underline{q}$ of length $len(q)/2$, representing the new demand vectors for the current request to compute the upper and lower bound, respectively. Afterwards, we look at all non-overlapping pairs of consecutive elements in q. To fill $\overline{q}$, we take the sum of the elements of each such pair, and to fill $\underline{q}$, we take the maximum element of each pair. At the end of the loop, we add $[r, \overline{q}]$ to $\overline{N}$ and $[r, \underline{q}]$ to $\underline{N}$.

Finally, we can compute an upper bound $\overline{Z}$ by calling the `recursive_sdp` function with T_d, the updated capacity vector $\overline{cap}$, and the new request pool $\overline{N}$. Similarly, we call `recursive_sdp` with T_d, $\underline{cap}$, and $\underline{N}$ to compute a lower bound $\underline{Z}$ for the DRAUSP. In the next section, we will evaluate the quality of these bounds and compare the runtime of the DC algorithm with the SDP algorithm.

Algorithm 3: A dimension compression algorithm for the DRAUSP

Function `get_bounds`(T_d, N, K, cap)**):**
  init empty sets $\underline{N}$ and $\overline{N}$
  if *K is not even* **then**
    append cap_K to cap
    $K \leftarrow K + 1$
  end
  init empty vectors $\overline{cap}$ and $\underline{cap}$ of length $K/2$
  for $i \leftarrow 1$ **to** $K/2$ **do**
    $\overline{cap}_i \leftarrow 2 \cdot \max\{cap_{2i-1}, cap_{2i}\}$
    $\underline{cap}_i \leftarrow \min\{cap_{2i-1}, cap_{2i}\}$
  end
  foreach $[r, q] \in N$ **do**
    if *len(q) is not even* **then**
      append 0 to q
    end
    init empty vectors $\overline{q}$ and $\underline{q}$ of length $len(q)/2$
    for $i \leftarrow 1$ **to** $len(q)/2$ **do**
      $\overline{q}_i \leftarrow q_{2i-1} + q_{2i}$
      $\underline{q}_i \leftarrow \max\{q_{2i-1}, q_{2i}\}$
    end
    add $[r, \overline{q}]$ to $\overline{N}$ and $[r, \underline{q}]$ to $\underline{N}$
  end
  $\overline{Z} =$ `recursive_sdp`$(T_d, \overline{cap}, \overline{N})$
  $\underline{Z} =$ `recursive_sdp`$(T_d, \underline{cap}, \underline{N})$
  return $\overline{Z}, \underline{Z}$

5 Computational Studies

In this section, we evaluate the quality of the bounds returned by the DC algorithm by comparing them with the optimal expected values returned by the SDP algorithm. In addition, we will assess the effectiveness of a simple FCFS heuristic that successively accepts stochastic incoming requests, if possible, and schedules them to the first possible time slot. Before presenting the results, we will describe the experimental environment.

5.1 Experimental Environment

In the revenue management literature, arrivals of requests and their demands are often assumed to be Poisson distributed [22]. We follow this assumption and generated each element of the demand vector q of any request $[r, q]$ in the finite pool of requests N using a Poisson process:

$$Pr(x) = \frac{e^{-\lambda_p} \lambda_p^x}{x!} \quad \forall x = 0, ..., \min_{1 \leq k \leq K} (C_k).$$

The parameter λ_p states the expected demand value in any time slot and also its variance. Within our experiments, we distinguish between small ($\lambda_p = 1$) and large ($\lambda_p = 3$) request demands. For each request $[r, q]$, we furthermore generated the revenue r by taking the quotient of 1 and an integer random number sampled from a discrete uniform distribution within the interval $[1, 1000]$.

For our experiments, we generated 51 problem instances with the following parameters: number of time slots $K \in \{4, 8, 12\}$, number of requests in the requests pool $|N| \in \{50, 1000\}$, expected demand value in any time slot $\lambda_p \in \{1, 3\}$, capacity per time slot $C \in \{10, 20\}$, and the number of incoming requests $T_d \in \{10, 20, 50\}$. In 21 of those instances, the length of the demand vectors $len(q)$ is equal to the number of time slots K, which means that the requests do not have to be scheduled, and we will only examine the accept/reject decisions for those instances. In the remaining instances, we consider $len(q) \in \{2, 3, 4, 6, 8\}$, with $len(q) < K$. The problem instances used in our experiments are available online (see https://doi.org/10.5281/zenodo.10640203).

We implemented the SDP algorithm and the DC algorithm in C++ and compiled them with GCC 13.1. To speed up the computation, we parallelized the SDP algorithm using OpenMP. All experiments were run on a shared compute cluster equipped with Intel Xeon Gold 6126 processors providing 24 CPU cores and 375 GB of RAM to each experiment. We enforced a time limit of 48 hours on each experiment.

5.2 Computational Results

In Table 1, we present the results for the instances where the length of the requests $len(q)$ is equal to K, i.e., without considering scheduling decisions in these instances. We can observe that the number of time slots K and the available capacity per time slot C are major factors influencing the runtime of the SDP algorithm. For instances with $K = 8$, the runtime quickly becomes unreasonably high when $\lambda_p = 1$. However, using demand vectors with higher demands by increasing λ_p to 3 reduces the runtime significantly because fewer requests can be accepted without exceeding the capacity. Despite the 375GB of RAM of our computing environment, we were not able to solve instances with $K = 12$, $|N| = 1000$, and $C = 20$, as not only the runtime but also the memory requirements grow exponentially with increasing C.

On average, the DC algorithm returns lower bounds that are within 14% from the optimal solution obtained by the SDP algorithm in about a tenth of the time. The upper bound exceeds the optimal solution by 9% on average. However, as with SDP, computing the upper bound quickly becomes intractable because, even though we halve the number of time slots, we double the available capacity to compute the upper bound. This problem is particularly evident for instances WC4-WC6, where the runtime to compute the upper bounds even exceeds the runtime of the SDP algorithm.

Let us define Δ^F_{SDP} as the relative improvement of an SDP solution (Z^*) compared to an FCFS solution (Z^F) of the same instance. As the number of

Table 1. Numerical results for the DRAUSP instances without the possibility of scheduling accepted requests.

Instance	FCFS	DCH				SDP		Δ^F_{SDP}
Id $(K, \lvert N\rvert, \lambda_p, C, T_d)$	Z^F	$\underline{Z}$	t	$\overline{Z}$	t	Z^*	t	%
WA1 (4, 50, 1, 10, 10)	3.80	3.87	0.01 s	4.63	0.02 s	4.33	0.15 s	14.0
WA2 (4, 50, 1, 10, 20)	4.85	5.41	0.02 s	6.90	0.03 s	6.43	0.43 s	32.6
WA3 (4, 50, 1, 10, 50)	5.82	7.48	0.06 s	10.08	0.07 s	9.49	1.06 s	63.1
WA4 (4, 50, 1, 20, 10)	4.56	5.02	0.02 s	5.08	0.04 s	5.07	0.90 s	11.2
WA5 (4, 50, 1, 20, 20)	8.23	8.20	0.04 s	9.61	0.09 s	9.23	4.60 s	12.2
WA6 (4, 50, 1, 20, 50)	10.45	12.39	0.08 s	16.05	0.24 s	15.36	18.1 s	47.0
WA7 (4, 1000, 1, 10, 10)	3.81	3.91	0.07 s	4.59	0.12 s	4.35	2.12 s	14.2
WA8 (4, 1000, 1, 10, 20)	4.90	5.53	0.15 s	7.04	0.30 s	6.58	4.23 s	34.3
WB1 (8, 50, 1, 10, 10)	3.46	3.50	0.08 s	4.51	0.93 s	4.08	261 s	18.0
WB2 (8, 50, 1, 10, 20)	4.01	4.54	0.25 s	6.30	3.06 s	5.47	1076 s	36.4
WB3 (8, 50, 1, 10, 50)	4.38	5.83	0.61 s	8.44	9.70 s	7.19	3567 s	64.2
WB4 (8, 50, 3, 10, 10)	1.18	1.55	0.03 s	1.95	0.08 s	1.69	0.07 s	43.2
WB5 (8, 50, 3, 10, 20)	1.27	1.86	0.03 s	2.38	0.17 s	2.08	0.11 s	63.8
WB6 (8, 50, 3, 20, 10)	2.45	2.91	0.22 s	3.57	2.83 s	3.22	33.4 s	31.4
WB7 (8, 50, 3, 20, 20)	2.65	3.60	0.63 s	4.48	8.13 s	4.06	132 s	53.2
WC1 (12, 50, 3, 10, 10)	1.02	1.45	0.02 s	1.83	0.06 s	1.54	0.03 s	51.0
WC2 (12, 50, 3, 10, 20)	1.07	1.65	0.03 s	2.19	0.16 s	1.78	0.07 s	66.4
WC3 (12, 50, 3, 10, 50)	1.12	1.84	0.06 s	2.54	0.37 s	2.13	0.14 s	90.2
WC4 (12, 1000, 3, 10, 10)	1.00	1.38	10.4 s	1.83	364 s	1.53	299 s	53.0
WC5 (12, 1000, 3, 10, 20)	1.05	1.65	22.3 s	2.19	901 s	1.79	855 s	70.5
WC6 (12, 1000, 3, 10, 50)	1.09	1.87	67.3 s	2.59	2393 s	2.17	2236 s	99.1
Average	3.44	4.07	4.88 s	5.18	176 s	4.74	404 s	46.1

incoming requests T_d and time slots K increases, while keeping all other parameters equal, we see that the value Δ^F_{SDP} becomes larger. The overall average of Δ^F_{SDP} for the instances in Table 1 is 46.1%.

The results of the experiments with $len(q) < k$, which require accepted requests to be scheduled onto the time slots, are shown in Table 2. We see that, again, the number of time slots K and the capacity per time slot C have strong effects on the runtime of the SDP algorithm. For example, doubling the number of time slots K from 4 to 8, without changing the other parameters, results in a 5000-fold increase in runtime for instances SA1 and SB1. The results of instances SB1 and SB2 show that, in some cases, an increasing number of incoming requests T_d can also lead to an exponentially increasing runtime. However, with an increasing length of demand vectors $len(q)$, the runtime decreases significantly, as there are fewer possibilities to schedule requests with longer demand vectors. Given our compute resources, we were unable to solve instances with more than 5 incoming requests when $K = 12$ and $len(q) = 4$.

On average, the DC algorithm finds a lower and upper bound in less than 0.08% and 1.5% of the runtime of the SDP algorithm, respectively. The obtained

Table 2. Numerical results for the DRAUSP instances with the possibility of scheduling accepted requests.

Instance	FCFS	DCH				SDP		Δ^F_{SDP}
Id $(K, len(q), \lvert N\rvert, \lambda_p, C, T_d)$	Z^F	$\underline{Z}$	t	$\overline{Z}$	t	Z^*	t	%
SA1 (4, 2, 50, 1, 10, 10)	3.43	5.03	0.03 s	5.08	0.06 s	5.08	0.42 s	48.1
SA2 (4, 2, 50, 1, 10, 20)	3.46	8.21	0.06 s	8.88	0.13 s	8.80	1.21 s	154.3
SA3 (4, 2, 50, 1, 10, 50)	3.48	12.99	0.20 s	13.52	0.42 s	13.44	3.53 s	286.2
SA4 (4, 3, 50, 1, 10, 10)	3.18	3.88	0.01 s	4.75	0.02 s	4.35	0.14 s	36.8
SA5 (4, 3, 50, 1, 10, 20)	3.20	5.07	0.02 s	6.70	0.05 s	5.97	0.29 s	86.6
SA6 (4, 3, 50, 1, 10, 50)	3.21	6.50	0.04 s	8.59	0.09 s	7.80	0.78 s	143.0
SA7 (4, 2, 50, 1, 20, 10)	4.53	5.08	0.05 s	5.08	0.11 s	5.08	1.41 s	12.1
SA8 (4, 2, 50, 1, 20, 20)	6.86	10.14	0.11 s	10.16	0.49 s	10.16	14.4 s	48.1
SA9 (4, 2, 50, 1, 20, 50)	6.86	18.95	0.27 s	20.46	2.01 s	20.38	60.3 s	197.1
SA10 (4, 3, 50, 1, 20, 10)	4.60	5.06	0.02 s	5.08	0.12 s	5.08	0.78 s	10.4
SA11 (4, 3, 50, 1, 20, 20)	6.44	8.16	0.05 s	9.74	0.13 s	9.02	3.37 s	40.1
SA12 (4, 3, 50, 1, 20, 50)	6.61	11.27	0.10 s	14.90	0.31 s	13.43	11.1 s	103.2
SA13 (4, 2, 1000, 1, 10, 10)	3.38	4.88	0.21 s	4.90	0.92 s	4.90	9.43 s	45.0
SA14 (4, 2, 1000, 1, 10, 20)	3.48	8.32	0.50 s	8.88	2.40 s	8.79	22.6 s	152.6
SB1 (8, 2, 50, 1, 10, 10)	3.46	5.08	1.59 s	5.08	10.2 s	5.08	2192 s	46.8
SB2 (8, 2, 50, 1, 10, 20)	3.48	10.14	5.92 s	10.16	103 s	10.16	42830 s	192.0
SB3 (8, 4, 50, 1, 10, 10)	3.17	5.03	0.45 s	5.08	3.87 s	5.08	1019 s	60.3
SB4 (8, 4, 50, 1, 10, 20)	3.18	8.16	1.65 s	8.94	14.0 s	8.76	4321 s	175.5
SB5 (8, 4, 50, 1, 10, 50)	3.18	12.01	3.45 s	12.97	45.1 s	12.60	14008 s	296.2
SB6 (8, 6, 50, 1, 10, 10)	3.09	4.26	0.21 s	4.55	1.30 s	4.44	102 s	43.7
SB7 (8, 6, 50, 1, 10, 20)	3.13	5.68	0.48 s	6.00	3.70 s	5.85	355 s	86.9
SB8 (8, 6, 50, 1, 10, 50)	3.16	7.52	1.41 s	7.55	10.9 s	7.53	1114 s	138.3
SB9 (8, 4, 50, 3, 10, 10)	1.26	3.04	0.33 s	3.73	3.50 s	3.38	685 s	168.3
SB10 (8, 4, 50, 3, 10, 20)	1.34	3.84	0.79 s	4.71	11.2 s	4.28	2126 s	219.4
SC1 (12, 4, 50, 3, 10, 5)	1.22	2.53	11.0 s	2.54	102 s	2.54	3368 s	108.2
SC2 (12, 6, 50, 3, 10, 5)	1.07	2.18	3.21 s	2.48	63 s	2.32	233 s	116.8
SC3 (12, 8, 50, 3, 10, 5)	1.02	1.53	0.94 s	1.98	10.4 s	1.74	5.78 s	70.6
SC4 (12, 8, 50, 3, 10, 10)	1.08	1.89	2.09 s	2.47	54.6 s	2.17	26.8 s	100.9
SC5 (12, 8, 50, 3, 10, 20)	1.14	2.24	5.13 s	2.85	160 s	2.55	72.2 s	123.7
SC6 (12, 8, 50, 3, 10, 50)	1.20	2.67	16.6 s	5.26	450 s	4.13	216 s	244.2
Average	3.26	6.38	1.90 s	7.10	35.1 s	6.83	2427 s	118.5

lower bound is within 7% of the optimal solution on average, and the upper bound exceeds it by about 4% on average.

With an average relative improvement Δ^F_{SDP} of over 118.5% for the instances with $len(q) < K$ (see Table 2), the performance of the FCFS heuristic becomes even worse than for instances with $len(q) = K$ (see Table 1). Again, the performance of the FCFS heuristic is mainly affected by the number of incoming requests T_d. However, in contrast to the results in Table 1, an increasing number of time slots K does not necessarily lead to a higher average Δ^F_{SDP}.

In conclusion, our experiments have shown that the scheduling decisions in the DRAUSP significantly increase the complexity of the problem. Nevertheless, the DC algorithm was still capable of finding good upper and lower bounds, which can be useful for evaluating the quality of heuristics. Our experiments have also shown that the FCFS heuristic is not sufficient to obtain satisfactory results, especially when acceptance and scheduling decisions are required, and thus more sophisticated heuristics are needed for the DRAUSP.

6 Conclusion and Future Research

In this paper, we studied the *Dynamic Request Acceptance and Unsplittable Scheduling Problem* (DRAUSP). We formulated the DRAUSP as a *Markov Decision Process* (MDP) and showed how *Stochastic Dynamic Programming* (SDP) can optimally solve this problem (see Algorithm 2). Furthermore, we proposed a *Dimension Compression* (DC) algorithm (see Algorithm 3) for the DRAUSP that provides lower and upper bounds to gauge the optimal solution of a DRAUSP instance.

Our experiments showed that the DC algorithm provides promising lower and upper bounds, and the runtime in doing so is relatively short in comparison to the runtime of the SDP algorithm. However, our experiments also demonstrated the complexity of the DRAUSP and showed that the FCFS heuristic leads to poor results. Therefore, future work should examine more sophisticated heuristic approaches, and since we provided an MDP model for the DRAUSP, *Reinforcement Learning* might be a promising direction.

References

1. Balseiro, S.R., Lu, H., Mirrokni, V.: The best of many worlds: Dual mirror descent for online allocation problems. Operations Research **71**(1), 101–119 (2023)
2. Bellman, R.E.: Dynamic Programming. Princeton University Press (1957)
3. Bondoux, N., Nguyen, A.Q., Fiig, T., Acuna-Agost, R.: Reinforcement learning applied to airline revenue management. Journal of Revenue and Pricing Management **19**(5), 332–348 (2020)
4. Chakaravarthy, V.T., Choudhury, A.R., Gupta, S., Roy, S., Sabharwal, Y.: Improved algorithms for resource allocation under varying capacity. Journal of Scheduling **21**, 313–325 (2018)
5. Gálvez, W., Grandoni, F., Ameli, A.J., Khodamoradi, K.: Approximation algorithms for demand strip packing. arXiv preprint arXiv:2105.08577 (2021)
6. Gosavi, A., Bandla, N., Das, T.: A reinforcement learning approach to airline seat allocation for multiple fare classes with overbooking" iie transactions, 34. Special issue on advances on large-scale optimization for logistics, production and manufacturing systems (2002)
7. Jahanjou, H., Kantor, E., Rajaraman, R.: Improved algorithms for scheduling unsplittable flows on paths. Algorithmica **85**(2), 563–583 (2023)
8. Kimes, S.E.: Yield management: a tool for capacity-considered service firms. Journal of operations management **8**(4), 348–363 (1989)

9. Kleywegt, A.J., Papastavrou, J.D.: The dynamic and stochastic knapsack problem. Operations research **46**(1), 17–35 (1998)
10. Kolen, A.W., Lenstra, J.K., Papadimitriou, C.H., Spieksma, F.C.: Interval scheduling: A survey. Naval Research Logistics (NRL) **54**(5), 530–543 (2007)
11. Mömke, T., Wiese, A.: A $(2+\epsilon)$-approximation algorithm for the storage allocation problem. In: International Colloquium on Automata, Languages, and Programming. pp. 973–984. Springer (2015)
12. Papastavrou, J.D., Rajagopalan, S., Kleywegt, A.J.: The dynamic and stochastic knapsack problem with deadlines. Management Science **42**(12), 1706–1718 (1996)
13. Pinedo, M.L.: Scheduling. Springer (2012)
14. Pruhs, K., Sgall, J., Torng, E.: Online scheduling. In: Leung, J., Kelly, L., Anderson, J.H. (eds.) Handbook of Scheduling: Algorithms, Models, and Performance Analysis. CRC Press (2004)
15. Puterman, M.L.: Markov decision processes: discrete stochastic dynamic programming. John Wiley & Sons (2014)
16. Schwind, M., Wendt, O.: Dynamic pricing of information products based on reinforcement learning: A yield-management approach. In: KI 2002: Advances in Artificial Intelligence. pp. 51–66. Springer (2002)
17. Seo, D.W., Chang, K., Cheong, T., Baek, J.G.: A reinforcement learning approach to distribution-free capacity allocation for sea cargo revenue management. Information Sciences **571**, 623–648 (2021)
18. Shihab, S.A.M., Logemann, C., Thomas, D.G., Wei, P.: Autonomous airline revenue management: A deep reinforcement learning approach to seat inventory control and overbooking. arXiv preprint arXiv:1902.06824 (2019)
19. Slotnick, S.A.: Order acceptance and scheduling: A taxonomy and review. European Journal of Operational Research **212**(1), 1–11 (2011)
20. Stork, F.: Stochastic resource-constrained project scheduling. Ph.D. thesis, TU Berlin (2001)
21. Talluri, K.T., Van Ryzin, G.: The theory and practice of revenue management. Springer (2004)
22. Weatherford, L.: The history of unconstraining models in revenue management. Journal of Revenue and Pricing Management **15**, 222–228 (2016)
23. Xu, L., Wang, Q., Huang, S.: Dynamic order acceptance and scheduling problem with sequence-dependent setup time. International Journal of Production Research **53**(19), 5797–5808 (2015)
24. Yang, L., Zeynali, A., Hajiesmaili, M.H., Sitaraman, R.K., Towsley, D.: Competitive algorithms for online multidimensional knapsack problems. Proceedings of the ACM on Measurement and Analysis of Computing Systems **5**(3), 1–30 (2021)